



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	APE 4 – Grafos: Mapa del Campus UTA
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	3 – “B”
Alumnos participantes:	Quispe Nasimba Daniela Alejandra
Asignatura:	Estructura de datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Implementar un grafo utilizando lista de adyacencia para representar dentro del Campus Huachi de la UTA y comparar los algoritmos BFS y Dijkstra.

Específicos:

- Diseñar e implementar un grafo mediante listas de adyacencia en Java para representar las diferentes ubicaciones y rutas existentes dentro del Campus Huachi de la Universidad Técnica de Ambato.
- Aplicar el algoritmo Breadth First Search (BFS) para determinar rutas con el menor número de paradas o nodos intermedios entre dos ubicaciones del campus.
- Implementar y analizar el algoritmo de Dijkstra para calcular la ruta de menor distancia considerando los pesos asociados a las conexiones del grafo y comparar sus resultados con los obtenidos mediante BFS.
- Comprender el funcionamiento de los grafos y las listas de adyacencia como estructuras de datos para representar conexiones.
- Comparar la eficiencia y los resultados obtenidos por BFS y Dijkstra, analizando sus ventajas según el criterio de búsqueda utilizado.

2.2 Modalidad

Presencial.

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

- Analizar el problema planteado e identificar las ubicaciones que serán representadas como nodos dentro del grafo del Campus Huachi de la Universidad Técnica de Ambato.
- Implementar la estructura del grafo utilizando listas de adyacencia para almacenar las conexiones entre los diferentes nodos.
- Completar los métodos marcados con TODO en el archivo APE4_Grafos.java: agregarNodo(), agregarArista(), bfs() y dijkstra().
- Crear los nodos correspondientes a las ubicaciones definidas en el programa y registrar sus conexiones mediante aristas con pesos asociados.
- Implementar el algoritmo Breadth First Search (BFS) para encontrar la ruta con el menor número de paradas entre un nodo de origen y un nodo de destino.
- Implementar el algoritmo de Dijkstra para determinar la ruta de menor distancia considerando los pesos asignados a las aristas.
- Ejecutar el programa y verificar el correcto funcionamiento de ambos algoritmos.



- Analizar y comparar los resultados obtenidos por BFS y Dijkstra, identificando las diferencias entre la ruta con menos paradas y la ruta con menor distancia.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC.
- Java
- Visual Studio Code
- GitHub
- Internet
- PC de escritorio o portátil con JDK instalado e IDE de su elección.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar

- Analizar la estructura y funcionamiento de los grafos como herramienta para representar rutas y conexiones entre diferentes ubicaciones.
- Diseñar la representación del Campus Huachi de la Universidad Técnica de Ambato mediante nodos y aristas dentro de un grafo.
- Implementar la estructura de datos utilizando listas de adyacencia para almacenar las conexiones entre los nodos del grafo.
- Desarrollar el método agregarNodo() para registrar las diferentes ubicaciones dentro del grafo.
- Implementar el método agregarArista() para establecer conexiones entre los nodos y asignar los pesos correspondientes a cada ruta.
- Programar el algoritmo Breadth First Search (BFS) para encontrar la ruta con el menor número de paradas entre dos ubicaciones del campus.
- Implementar el algoritmo de Dijkstra para determinar la ruta de menor distancia considerando el peso de las aristas.
- Ejecutar pruebas de funcionamiento para verificar la correcta creación de nodos, aristas y recorridos dentro del grafo.
- Comparar los resultados obtenidos por los algoritmos BFS y Dijkstra, analizando las diferencias entre una ruta con menos paradas y una ruta con menor distancia.
- Documentar el código mediante comentarios descriptivos que faciliten la comprensión de la lógica implementada.
- Generar evidencias de ejecución mediante capturas de pantalla de los resultados obtenidos en consola.
- Elaborar el informe técnico con el desarrollo de la práctica, resultados obtenidos, análisis comparativo y conclusiones.

2.7 Resultados obtenidos

Link de GitHub:

https://github.com/Daniela-Quispe/APE-4_Grafos



Grafos: Mapa del Campus UTA

1. INTRODUCCION:

Las estructuras de datos constituyen un elemento fundamental dentro del desarrollo de software, ya que permiten organizar, almacenar y gestionar información de manera eficiente. Entre las estructuras más utilizadas para representar relaciones y conexiones entre diferentes elementos se encuentran los grafos, los cuales son ampliamente empleados en aplicaciones relacionadas con redes de comunicación, sistemas de transporte, navegación, redes sociales, inteligencia artificial y optimización de rutas.

Un grafo está compuesto por un conjunto de nodos o vértices y un conjunto de aristas que representan las conexiones existentes entre ellos. Gracias a esta estructura es posible modelar escenarios reales donde se requiere analizar la conectividad entre distintos puntos y determinar rutas óptimas de desplazamiento. Dependiendo del problema a resolver, pueden emplearse diferentes algoritmos de recorrido y búsqueda, cada uno con características y objetivos específicos. En aplicaciones reales, los grafos permiten modelar redes de transporte, mapas geográficos, sistemas de navegación GPS, redes sociales, telecomunicaciones, sistemas de recomendación y estructuras jerárquicas complejas.

En la presente práctica se desarrolló un grafo utilizando listas de adyacencia para representar las rutas existentes dentro del Campus Huachi de la Universidad Técnica de Ambato. Esta representación permite almacenar de manera eficiente las conexiones entre las diferentes ubicaciones consideradas dentro del modelo, facilitando la aplicación de algoritmos de búsqueda sobre la estructura creada.

Además, se implementaron dos algoritmos fundamentales para la búsqueda de rutas:

- Breadth First Search (BFS).
- Algoritmo de Dijkstra.

Cada uno de estos algoritmos resuelve un problema diferente:

- BFS encuentra la ruta con menor cantidad de paradas.
- Dijkstra encuentra la ruta con menor distancia acumulada.

Finalmente, esta práctica contribuye al fortalecimiento de los conocimientos relacionados con grafos, listas de adyacencia, algoritmos de recorrido y búsqueda de caminos mínimos, permitiendo aplicar conceptos teóricos de estructuras de datos en un contexto práctico relacionado con la representación de rutas dentro de un entorno universitario.

2. MARCO TEORICO:

Grafos:

Los grafos son estructuras de datos no lineales utilizadas para representar relaciones entre diferentes elementos. Matemáticamente, un grafo está compuesto por un conjunto de vértices o nodos y un conjunto de aristas que conectan dichos vértices. Esta estructura permite modelar situaciones reales donde existen conexiones entre distintos puntos, como redes de transporte, redes informáticas, sistemas de navegación, mapas geográficos y redes sociales.

Un grafo puede representarse mediante la expresión: $[G = (V, E)]$ donde:

- (V) representa el conjunto de vértices o nodos.
- (E) representa el conjunto de aristas o conexiones.



En esta práctica, los nodos representan ubicaciones dentro del Campus Huachi de la Universidad Técnica de Ambato, mientras que las aristas representan las rutas que conectan dichas ubicaciones.

Tipos de Grafos:

Los grafos pueden clasificarse según las características de sus conexiones:

➤ **Grafo Dirigido:**

Es aquel en el que las aristas tienen una dirección específica. Esto significa que existe un origen y un destino definidos para cada conexión.

Ejemplo:

$A \rightarrow B$

En este caso se puede ir de A hacia B, pero no necesariamente de B hacia A.

➤ **Grafo No Dirigido:**

Es aquel en el que las conexiones pueden recorrerse en ambos sentidos.

Ejemplo:

$A \leftrightarrow B$

En esta práctica se utilizó un grafo no dirigido, ya que las rutas entre los nodos pueden recorrerse en ambas direcciones.

➤ **Grafo Ponderado:**

Es un grafo en el que cada arista posee un peso asociado. Dicho peso puede representar distancia, tiempo, costo o cualquier otro valor cuantificable.

Ejemplo:

$A \text{ ----}50\text{ ----} B$

Los pesos utilizados en esta práctica representan distancias entre las distintas ubicaciones del campus.

Lista de Adyacencia:

La lista de adyacencia es una técnica utilizada para representar grafos de forma eficiente. Consiste en almacenar para cada nodo una lista de todos los nodos con los que se encuentra conectado.

Ejemplo:

Universidad:

FISEI (50)

Comedor (20)

FISEI:

Universidad (50)

Idiomas (40)

Entre sus principales ventajas se encuentran:

- Menor consumo de memoria en comparación con la matriz de adyacencia.
- Mayor eficiencia para grafos con pocas conexiones.
- Facilidad para recorrer los vecinos de un nodo.

Algoritmo Breadth First Search (BFS):



Breadth First Search (BFS), conocido como búsqueda en anchura, es un algoritmo de recorrido de grafos que explora los nodos por niveles. Su funcionamiento se basa en una cola FIFO (First In, First Out), donde los primeros elementos en ingresar son los primeros en salir.

El algoritmo inicia desde un nodo origen y visita todos sus vecinos directos antes de continuar con los siguientes niveles del grafo.

Proceso general:

1. Se agrega el nodo inicial a una cola.
2. Se marca como visitado.
3. Se recorren todos sus vecinos.
4. Los vecinos no visitados se agregan a la cola.
5. El proceso continúa hasta encontrar el nodo destino o recorrer todo el grafo.

La principal característica de BFS es que encuentra la ruta con el menor número de paradas o nodos intermedios, sin considerar la distancia o peso de las conexiones.

Su complejidad temporal es: $[O(V + E)]$ donde:

- (V) es el número de vértices.
- (E) es el número de aristas.

Algoritmo de Dijkstra:

El algoritmo de Dijkstra es uno de los métodos más utilizados para resolver problemas de caminos mínimos en grafos ponderados. Su objetivo es determinar la ruta de menor costo o menor distancia entre un nodo origen y los demás nodos del grafo. A diferencia de BFS, este algoritmo considera el peso asociado a cada arista.

El procedimiento general consiste en:

1. Asignar distancia infinita a todos los nodos.
2. Asignar distancia cero al nodo inicial.
3. Seleccionar el nodo con menor distancia conocida.
4. Actualizar las distancias de sus nodos vecinos.
5. Repetir el proceso hasta recorrer todo el grafo.

Para optimizar su funcionamiento se utiliza una cola de prioridad, permitiendo seleccionar rápidamente el nodo con la menor distancia acumulada.

Su complejidad temporal es: $[O((V + E)\log V)]$ cuando se implementa mediante una cola de prioridad.

La principal ventaja de Dijkstra es que garantiza encontrar la ruta de menor distancia en grafos con pesos positivos.

Cola (Queue):

Una cola es una estructura de datos lineal que sigue el principio FIFO (First In, First Out), es decir, el primer elemento que entra es el primero que sale.

Las operaciones principales son:

- Encolar (Insertar).
- Desencolar (Eliminar).
- Consultar el primer elemento.

En esta práctica, BFS utiliza una cola para gestionar el recorrido de los nodos por niveles.

Cola de Prioridad (Priority Queue):



Una cola de prioridad es una estructura de datos especializada que permite extraer siempre el elemento con mayor prioridad. En el caso del algoritmo de Dijkstra, la prioridad está determinada por la menor distancia acumulada.

Gracias a esta estructura, el algoritmo puede seleccionar de manera eficiente el siguiente nodo que debe procesarse.

Aplicación de los Grafos en Sistemas Reales:

Los grafos tienen una amplia variedad de aplicaciones en el ámbito tecnológico y científico.

Algunas de las más importantes son:

- Sistemas de navegación GPS.
- Redes de transporte urbano.
- Redes sociales.
- Redes de telecomunicaciones.
- Sistemas de recomendación.
- Inteligencia artificial.
- Diseño de circuitos.
- Organización de rutas logísticas.

En esta práctica, los grafos se aplican para representar rutas entre ubicaciones del Campus Huachi de la Universidad Técnica de Ambato, permitiendo analizar diferentes alternativas de recorrido mediante algoritmos de búsqueda y optimización.

3. DESARROLLO DEL EJERCICIO:

3.1 ANALISIS DEL PROBLEMA:

La práctica consiste en implementar un grafo mediante listas de adyacencia para representar rutas dentro del Campus Huachi de la Universidad Técnica de Ambato y posteriormente comparar los algoritmos BFS y Dijkstra.

Para ello fue necesario identificar:

- Los nodos del grafo.
- Las conexiones entre nodos.
- Las distancias asociadas a cada conexión.
- Los algoritmos que permitirán encontrar rutas.

3.2 CREACION DE LA CLASE NODO:

Se creó la clase Nodo, cuya función es representar cada ubicación del campus.

```
static class Nodo {  
    String id;  
    String nombre;  
  
    public Nodo(String id, String nombre) {  
        this.id = id;  
        this.nombre = nombre;  
    }  
}
```

Esta clase almacena:

Atributo	Descripción
id	Identificador único



nombre	Nombre de la ubicación
--------	------------------------

Ejemplo:

```
grafo.agregarNodo("uta", "Universidad");
```

Función: Representa el nodo Universidad.

3.3 CREACION DE LA CLASE ARISTA:

Se creó la clase Arista, encargada de representar las conexiones entre nodos.

```
static class Arista {  
    String destino;  
    int peso;  
  
    public Arista(String destino, int peso) {  
        this.destino = destino;  
        this.peso = peso;  
    }  
}
```

Permite almacenar:

Atributo	Descripción
destino	Nodo conectado
peso	Distancia de la ruta

Ejemplo:

```
new Arista("fisei", 50);
```

Función: Indica una conexión hacia FISEI con distancia 50.

3.4 CREACION DE LA ESTRUCTURA GRAFO:

Se creó la clase principal encargada de almacenar todos los nodos y conexiones.

```
Map<String, Nodo> nodos = new HashMap<>();  
Map<String, List<Arista>> adyacencia = new HashMap<>();
```

Función de cada estructura:

Mapa de nodos:

```
Map<String, Nodo> nodos
```

Función: Permite acceder rápidamente a cualquier nodo utilizando su identificador.

Ejemplo:

```
nodos.get("uta");
```

Función: Retorna el nodo Universidad.

Lista de adyacencia:



Map<String, List<Arista>> adyacencia

Función: Almacena las conexiones existentes entre nodos.

Ejemplo:

```
uta
├── fisei (50)
└── comedor (20)
```

3.5 IMPLEMENTACION DEL METODO AGREGARNODO():

Se implementó el método encargado de registrar nuevas ubicaciones.

```
public void agregarNodo(String id, String nombre) {
    nodos.put(id, new Nodo(id, nombre));
    adyacencia.put(id, new ArrayList<>());
}
```

Funcionamiento de cada estructura:

Crear el nodo:

new Nodo(id, nombre)

Guardarlo en el mapa:

nodos.put(...)

Crear lista vacía de conexiones:

adyacencia.put(...)

3.6 IMPLEMENTACION DEL METODO AGREGARARISTA():

Este método permite conectar dos nodos.

```
public void agregarArista(
    String origen,
    String destino,
    int peso) {

    adyacencia.get(origen).add(new Arista(destino, peso));
    adyacencia.get(destino).add(new Arista(origen, peso));
}
```

Explicación: El grafo es no dirigido. Por esta razón: uta → fisei; también implica: fisei → uta.

Ejemplo:

grafo.agregarArista("uta", "fisei", 50);

Resultado:

Universidad ↔ FISEI

Distancia = 50

3.7 REGISTRO DE NODOS:

Se agregaron los nodos definidos en la práctica:



```
grafo.agregarNodo("uta", "Universidad");  
grafo.agregarNodo("fisei", "FISEI");  
grafo.agregarNodo("idiomas", "Idiomas");  
grafo.agregarNodo("biblioteca", "Biblioteca");  
grafo.agregarNodo("estadio", "Estadio");  
grafo.agregarNodo("comedor", "Comedor");
```

Grafo obtenido:

- Universidad
- FISEI
- Idiomas
- Biblioteca
- Estadio
- Comedor

Total: 6 nodos.

3.8 REGISTRO DE CONEXIONES:

Se registraron las aristas:

```
grafo.agregarArista("uta", "fisei", 50);  
grafo.agregarArista("fisei", "idiomas", 40);  
grafo.agregarArista("idiomas", "biblioteca", 30);  
grafo.agregarArista("biblioteca", "estadio", 70);  
grafo.agregarArista("uta", "comedor", 20);  
grafo.agregarArista("comedor", "estadio", 200);
```

Representación gráfica:

Universidad

|
50

|
FISEI

|
40

|
Idiomas

|
30

|
Biblioteca

|
70

|
Estadio

Universidad

|
20

|
Comedor

|
200



|
Estadio

3.9 IMPLEMENTACION DEL ALGORITMO BFS:

El algoritmo BFS utiliza: *Queue<List<String>> cola*; para recorrer el grafo por niveles.

Funcionamiento:

- Se crea un camino inicial.
- Se agrega a la cola.
- Se marca el nodo como visitado.
- Se exploran los vecinos.
- Se generan nuevos caminos.
- Se retorna la primera ruta que alcance el destino.

Se desarrolló el método:

```
public List<String> bfs(String inicio,String fin)
```

Función: Encontrar la ruta con menor cantidad de paradas.

Inicialización:

```
Queue<List<String>> cola = new LinkedList<>();
```

Función: Se crea una cola para recorrer el grafo.

```
Set<String> visitados = new HashSet<>();
```

Función: Se almacenan los nodos visitados.

```
List<String> caminoInicial = new ArrayList<>();
```

Función: Se crea la ruta inicial.

Agregar nodo inicial:

```
caminoInicial.add(inicio);
```

Resultado: [uta]

Agregar a la cola:

```
cola.add(caminoInicial);
```

Resultado: [[uta]]

Recorrido BFS:

Primer recorrido: uta. Vecinos: fisei, comedor.

Cola: [uta,fisei] [uta,comedor]

Segundo recorrido:

uta,fisei

Función: Se agregan nuevos vecinos.

Tercer recorrido:

uta,comedor



Vecino encontrado: estadio

Ruta encontrada: uta → comedor → estadio

Resultado BFS: Universidad → Comedor → Estadio

Número de saltos: 2

3.10 IMPLEMENTACION DEL ALGORITMO DIJKSTRA:

El algoritmo utiliza:

Map<String,Integer> distancias

Map<String,String> anteriores

PriorityQueue<String> cola

Distancias: Almacenan el costo mínimo conocido.

Anteriores: Permiten reconstruir la ruta final.

Cola de Prioridad: Selecciona siempre el nodo con menor distancia acumulada.

Se implementó el método:

```
public List<String> dijkstra(String inicio,String fin)
```

Objetivo: Encontrar la ruta de menor distancia.

Inicialización de distancias:

```
distancias.put(nodo,Integer.MAX_VALUE);
```

Resultado inicial:

Nodo	Distancia
uta	0
fisei	∞
idiomas	∞
biblioteca	∞
estadio	∞
comedor	∞

Evaluación de vecinos:

Desde Universidad:

FISEI = 50

Comedor = 20

Desde FISEI:

Idiomas = 90

Desde Idiomas:

Biblioteca = 120

Desde Biblioteca:

Estadio = 190

Comparación de rutas:

Ruta 1:

Universidad → Comedor → Estadio

Costo: 20 + 200 = 220



Ruta 2:

Universidad → FISEI → Idiomas → Biblioteca → Estadio

Costo: $50 + 40 + 30 + 70 = 190$

Ruta seleccionada:

Dijkstra escoge:

➤ Universidad → FISEI → Idiomas → Biblioteca → Estadio

Ya que: $190 < 220$

3.11 EJECUCION DEL PROGRAMA:

Al ejecutar el programa se obtuvo:

===== BFS =====

Universidad (uta) -> Comedor (comedor) -> Estadio (estadio)

===== DIJKSTRA =====

Universidad (uta) -> FISEI (fisei) -> Idiomas (idiomas) -> Biblioteca (biblioteca) -> Estadio (estadio)

3.12 ANALISIS DE RESULTADOS:

BFS encontró la ruta con menor número de paradas:

Universidad → Comedor → Estadio

Distancia: 220

Dijkstra encontró la ruta con menor distancia:

Universidad → FISEI → Idiomas → Biblioteca → Estadio

Distancia: 190

Por lo tanto, se comprobó que ambos algoritmos generan resultados diferentes porque optimizan criterios distintos. BFS minimiza el número de saltos, mientras que Dijkstra minimiza la distancia total recorrida.

4. COMPARACION DE RESULTADOS:

Una vez ejecutados los algoritmos BFS y Dijkstra sobre el grafo que representa las rutas del Campus Huachi, se observaron diferencias importantes en los caminos obtenidos debido a que cada algoritmo optimiza un criterio distinto.

Resultado obtenido con BFS:

Ruta encontrada: Universidad (uta) → Comedor (comedor) → Estadio (estadio)

Número de paradas: 2 conexiones

Distancia total: $20 + 200 = 220$

Análisis de BFS:

El algoritmo BFS (Breadth-First Search) busca la ruta con la menor cantidad de nodos intermedios entre el origen y el destino. No considera el peso o distancia de las aristas, únicamente la cantidad de conexiones necesarias para llegar al destino.

Por esta razón, seleccionó la ruta: Universidad → Comedor → Estadio; ya que requiere únicamente dos conexiones para llegar al estadio.

Resultado obtenido con Dijkstra:

Ruta encontrada: Universidad (uta) → FISEI (fisei) → Idiomas (idiomas) → Biblioteca (biblioteca) → Estadio (estadio)

Distancia total: $50 + 40 + 30 + 70 = 190$



Análisis de Dijkstra:

El algoritmo de Dijkstra busca la ruta con el menor costo acumulado considerando los pesos de las aristas. Aunque la ruta encontrada contiene más nodos intermedios, la distancia total recorrida es menor.

Por ello seleccionó: Universidad → FISEI → Idiomas → Biblioteca → Estadio; porque su costo total es de 190 unidades, valor inferior a los 220 de la ruta encontrada por BFS.

Tabla Comparativa:

Criterio	BFS	Dijkstra
Objetivo	Menor número de paradas	Menor distancia total
Considera pesos	No	Sí
Ruta encontrada	Universidad → Comedor → Estadio	Universidad → FISEI → Idiomas → Biblioteca → Estadio
Número de conexiones	2	4
Distancia total	220	190
Resultado óptimo según distancia	No	Sí
Estructura utilizada	Cola	Cola de prioridad
Complejidad	$O(V+E)$	$O((V+E)\log V)$

Los resultados demuestran que una ruta con menos paradas no necesariamente es la más corta en distancia. BFS prioriza llegar al destino utilizando la menor cantidad de conexiones posibles, mientras que Dijkstra analiza todas las alternativas disponibles para determinar el camino con el menor costo acumulado.

En aplicaciones reales como sistemas de navegación, transporte o movilidad dentro de un campus universitario, Dijkstra suele ser más adecuado cuando se desea minimizar la distancia o el tiempo de recorrido. Por otro lado, BFS resulta útil cuando todas las conexiones tienen el mismo costo o cuando interesa minimizar únicamente el número de pasos necesarios para llegar al destino.

Conclusión de la comparación:

Se verificó correctamente el comportamiento de ambos algoritmos:

BFS encontró la ruta con menos paradas.

Dijkstra encontró la ruta con menor distancia total.

Los resultados obtenidos coinciden con la teoría estudiada sobre grafos y algoritmos de búsqueda de caminos.

5. CAPTURAS DE LA SALIDA:

a) Consola ejecutando el programa:

```
C:\Users\HP> cmd /C "C:\Program Files\Java\jdk-21.0.11\bin\java.exe" -XX:+ShowCodeDetailsInExceptionMessages -cp C:\Users\HP\AppData\Local\Temp\vscodesws_190e0\jdt_ws\jdt.ls-java-project\bin APE4_Grafos "
===== BFS =====
Universidad (uta) -> Comedor (comedor) -> Estadio (estadio)

===== DIJKSTRA =====
Universidad (uta) -> FISEI (fisei) -> Idiomas (idiomas) -> Biblioteca (biblioteca) -> Estadio (estadio)
```

b) Resultado del algoritmo BFS:

```
===== BFS =====
Universidad (uta) -> Comedor (comedor) -> Estadio (estadio)
```



c) Resultado del algoritmo Dijkstra:

```
===== DIJKSTRA =====  
Universidad (uta) -> FISEI (fisei) -> Idiomas (idiomas) -> Biblioteca (biblioteca) -> Estadio (estadio)
```

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- Se implementó correctamente un grafo utilizando listas de adyacencia para representar las conexiones entre diferentes ubicaciones del Campus Huachi de la Universidad Técnica de Ambato. Esta estructura permitió modelar de manera eficiente las rutas existentes entre los distintos nodos, optimizando el almacenamiento de las conexiones y facilitando la aplicación de algoritmos de búsqueda.
- La implementación del algoritmo BFS permitió identificar la ruta con el menor número de paradas entre el nodo de origen y el nodo destino. Durante las pruebas se comprobó que BFS recorre el grafo por niveles y encuentra rápidamente caminos con menos conexiones, sin considerar la distancia o el peso asociado a cada ruta.
- La aplicación del algoritmo de Dijkstra permitió determinar la ruta con el menor costo acumulado considerando los pesos de las aristas. Los resultados demostraron que una ruta con más nodos intermedios puede ser más eficiente cuando la distancia total recorrida es menor, evidenciando la importancia de considerar los pesos en problemas de optimización de rutas.
- La comparación entre BFS y Dijkstra permitió comprender que ambos algoritmos tienen objetivos diferentes. Mientras BFS busca minimizar la cantidad de conexiones necesarias para llegar al destino, Dijkstra busca minimizar la distancia total recorrida. Por esta razón, ambos algoritmos pueden generar rutas distintas para un mismo problema.
- La práctica fortaleció los conocimientos sobre grafos, listas de adyacencia, estructuras dinámicas, colas, mapas y algoritmos de búsqueda de caminos. Además, permitió relacionar conceptos teóricos de estructuras de datos con aplicaciones reales, demostrando cómo estas técnicas pueden utilizarse para resolver problemas de navegación y planificación de rutas dentro de entornos universitarios y sistemas informáticos de mayor complejidad.

2.10 Recomendaciones

- Se recomienda continuar practicando la implementación de grafos utilizando diferentes estructuras de representación, como matrices de adyacencia y listas de adyacencia, con el fin de comprender las ventajas y limitaciones de cada método según el tipo de problema a resolver.
- Es recomendable realizar pruebas con grafos de mayor tamaño y complejidad, incorporando más nodos y conexiones dentro del Campus Huachi, para analizar el



comportamiento y rendimiento de los algoritmos BFS y Dijkstra en escenarios más cercanos a situaciones reales.

- Se recomienda complementar el estudio de BFS y Dijkstra con otros algoritmos de grafos, como Depth-First Search (DFS), Floyd-Warshall y A*, para ampliar los conocimientos sobre búsqueda y optimización de caminos.
- Es importante documentar adecuadamente el código mediante comentarios claros y descriptivos, ya que esto facilita el mantenimiento, la comprensión de la lógica implementada y el trabajo colaborativo en proyectos de desarrollo de software.
- Se recomienda utilizar datos reales de distancias y ubicaciones del Campus Huachi en futuras implementaciones, con el propósito de obtener resultados más precisos y desarrollar aplicaciones prácticas orientadas a la navegación, planificación de recorridos y gestión de espacios universitarios.

2.11 Referencias bibliográficas

- [1] Brigham Young University, “Dijkstra's Algorithm,” s.f. [En línea]. Disponible en: <https://labs.acme.byu.edu/Volume2/Dijkstra/Dijkstra.html> [Accedido: 30-may-2026].
- [2] Codecademy, “Dijkstra's Shortest Path Algorithm,” s.f. [En línea]. Disponible en: <https://www.codecademy.com/article/dijkstras-shortest-path-algorithm> [Accedido: 30-may-2026].
- [3] freeCodeCamp, “Algoritmo de la ruta más corta de Dijkstra: Introducción gráfica,” s.f. [En línea]. Disponible en: <https://www.freecodecamp.org/espanol/news/algoritmo-de-la-ruta-mas-corta-de-dijkstra-introduccion-grafica/> [Accedido: 30-may-2026].
- [4] Interview Cake, “Breadth First Search (BFS),” s.f. [En línea]. Disponible en: <https://www.interviewcake.com/concept/java/bfs> [Accedido: 30-may-2026].
- [5] Memgraph, “Deep Path Traversal,” s.f. [En línea]. Disponible en: <https://memgraph.com/docs/advanced-algorithms/deep-path-traversal-> [Accedido: 30-may-2026].
- [6] Wikipedia, “Algoritmo de Dijkstra,” 2026. [En línea]. Disponible en: https://es.wikipedia.org/wiki/Algoritmo_de_Dijkstra [Accedido: 30-may-2026].
- [7] Wikipedia, “Búsqueda en anchura,” 2026. [En línea]. Disponible en: https://es.wikipedia.org/wiki/B%C3%BAsqueda_en_anchura [Accedido: 30-may-2026].
- [8] W3Schools, “Dijkstra's Algorithm,” s.f. [En línea]. Disponible en: https://www.w3schools.com/dsa/dsa_algo_graphs_dijkstra.php [Accedido: 30-may-2026].

2.12 Anexos

GitHub Personal:

